

# Inteligencia Artificial y Neurociencias

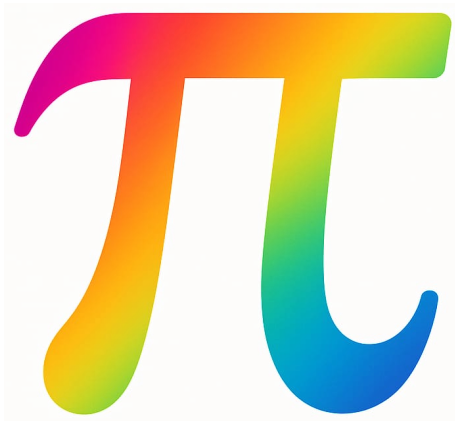
Licenciatura en Tecnología Digital, UTDT, 2S-2026

**Profesores:** Agustín Gravano, Andrés Rieznik

**Auxiliares:** Gerardo Del Toro, Antonella Giordano Furchi

Temas Avanzados: Gradiente de Políticas;  
Monte Carlo Tree Search; Preguntas para Neurociencias

# Métodos de gradiente de políticas



# Métodos de gradiente de políticas

Los métodos que vimos buscan aproximar una función de valor  $q_*(s, a)$ . De esa manera, intentan aprender una buena política.

En algunos problemas, puede resultar más sencillo aproximar una política que una función de valor, porque hay demasiadas acciones disponibles, o incluso acciones continuas.

Antes:  $q^*(s, a)$  política

EJEMPLOS: operación de redes eléctricas; sistemas de recomendación; trading algorítmico; operación de brazos robóticos; LLMs.

Los métodos de gradiente de políticas buscan aprender una política parametrizada que pueda seleccionar acciones sin necesidad de una función de valor:

Devuelve las probabilidades de cada acción

$$\pi(a | s, \theta) = \Pr \{A_t = a | S_t = s, \theta_t = \theta\}$$

donde  $\theta \in \mathbb{R}^{d'}$  es el vector de parámetros (pesos) de la política.

Si definimos una métrica de desempeño  $J(\theta)$ , podemos actualizar  $\theta$  en forma iterativa con ascenso por el gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

# Algoritmo REINFORCE para estimar $\pi_*$

$\pi : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^{d'} \rightarrow [0, 1]$  es una función parametrizable  
con pesos  $\theta \in \mathbb{R}^{d'}$  (ej.: red neuronal)

Inicializar  $\theta \in \mathbb{R}^{d'}$  arbitrariamente

Repetir:

Generar un episodio según  $\pi$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$ .

Para cada paso del episodio,  $t = 0, 1, \dots, T-1$ :

$$G \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$$

$$\theta \leftarrow \theta + \alpha G \nabla \ln \pi(A_t | S_t, \theta)$$

MC

## OBSERVACIONES:

- En la actualización de  $\theta$ , ya no buscamos mejorar una estimación de  $q_*(s, a)$ .
- En cambio, ajustamos directamente la probabilidad de elegir  $A_t$  en  $S_t$ :  $\pi(A_t | S_t, \theta)$  sube o baja proporcionalmente al retorno observado  $G$ .
- REINFORCE vendría a ser un método MC de gradiente de políticas. Actor-Critic (próxima slide) sería una variante TD.

# Métodos Actor-Critic

Bootstrapping = estimar un valor usando otras estimaciones tuyas, en lugar de esperar el resultado real completo.

Actualización de REINFORCE:  $\theta_{t+1} \doteq \theta_t + \alpha G_t \nabla \ln \pi(A_t | S_t, \theta_t)$

Si en lugar de medir  $G_t$  (como en MC), hacemos bootstrapping (como en TD), obtenemos la actualización de los métodos *actor-critic*:

$$\theta_{t+1} \doteq \theta_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \cdot \nabla \ln \pi(A_t | S_t, \theta_t)$$

donde:

$\pi : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^{d'} \rightarrow [0, 1]$  es una red con pesos  $\theta \in \mathbb{R}^{d'}$  **Actor**

$\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$  es una red con pesos  $\mathbf{w} \in \mathbb{R}^d$  **Crítico**

( $\mathbf{w}$  también se actualiza en cada iteración, en forma similar a DQL)

Notar que el **objetivo** es el error TD: la diferencia entre la nueva estimación del valor de  $S_t$  (usando  $R_{t+1}$  y  $S_{t+1}$ ) y la vieja estimación del valor de  $S_t$ .

Las redes  $\pi$  y  $\hat{v}$  se conocen como **actor** y **crítico**, respectivamente.

**Proximal Policy Optimization (PPO)** extiende Actor-Critic restringiendo cuánto puede cambiar  $\pi$  en cada update: logra un aprendizaje más estable sin sacrificar eficiencia. PPO es el estado del arte en muchos dominios.

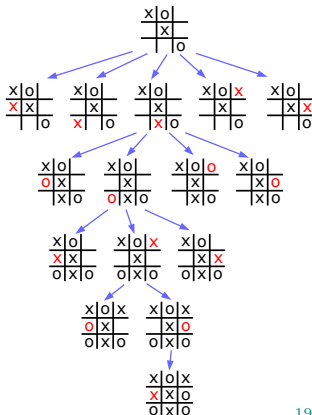
# Monte Carlo Tree Search (MCTS)



# Planificación clásica para juegos

## Ta-te-ti

Árbol de búsqueda:



El algoritmo Minimax  
resuelve este problema.✓



1997:

**Ajedrez:** ~30 movidas por posición; duración: ~70 movidas (35x2);  
hay buenas heurísticas de evaluación para podar la búsqueda. ✓

**Go:** ~250 movidas por posición; dur.: ~200 movidas (100x2); no se  
conocen buenas heurísticas de evaluación para podar la búsqueda. ✗

# DIFFICULTY OF VARIOUS GAMES FOR COMPUTERS

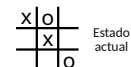
EASY

|                                                                                 |                                         |                                                                                                                                                             |
|---------------------------------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SOLVED<br>COMPUTERS CAN<br>PLAY PERFECTLY                                       | SOLVED FOR<br>ALL POSSIBLE<br>POSITIONS | <p>TIC-TAC-TOE</p> <p>NIM</p> <p>GHOST (1989)</p> <p>CONNECT FOUR (1995)</p>                                                                                |
|                                                                                 | SOLVED FOR<br>STARTING<br>POSITIONS     | <p>GOMOKU</p> <p>CHECKERS (2007)</p>                                                                                                                        |
| COMPUTERS CAN<br>BEAT TOP HUMANS                                                |                                         | <p>SCRABBLE</p> <p>COUNTERSTRIKE</p> <p>REVERSI</p> <p>BEER PONG (WUC ROBOT)</p>                                                                            |
|                                                                                 |                                         | <p>CHESS</p> <p>FEBRUARY 10, 1996:<br/>FIRST WIN BY COMPUTER<br/>AGAINST TOP HUMAN<br/>NOVEMBER 21, 2005<br/>LAST WIN BY HUMAN<br/>AGAINST TOP COMPUTER</p> |
| COMPUTERS STILL<br>LOSE TO TOP HUMANS<br>(BUT FOCUSED R&D<br>COULD CHANGE THIS) |                                         | <p>JEOPARDY!</p> <p>STARCRRAFT</p> <p>POKER</p>                                                                                                             |
|                                                                                 |                                         | <p>ARIMAA</p> <p>GO</p>                                                                                                                                     |
| COMPUTERS<br>MAY NEVER<br>OUTPLAY HUMANS                                        |                                         | <p>MAO</p> <p>SNAKES AND LADDERS</p>                                                                                                                        |
|                                                                                 |                                         | <p>SEVEN MINUTES<br/>IN HEAVEN</p> <p>CALVINBALL</p>                                                                                                        |

HARD

<https://xkcd.com/1002/>  
11 de enero de 2012

# Trayectorias simuladas



Una **trayectoria simulada** va desde el estado actual hasta el final del episodio.

Las acciones se eligen según una **política** dada.

Las recompensas y los estados siguientes se aproximan mediante un **modelo del ambiente**.

El agente no interactúa con el ambiente real.

Solo **simula** (imagina) posibles continuaciones del episodio a partir del estado actual, usando su propio modelo del ambiente.

# Decision-time planning: Métodos Rollout

1. Simular una trayectoria comenzando con una acción  $A$  desde el estado actual  $S$ .
2. Al llegar a un estado terminal, actualizar solamente la estimación del valor de  $(S, A)$ .
3. Repetir.

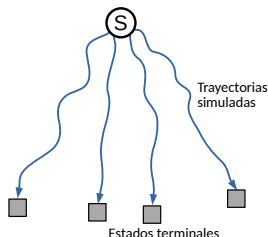
Luego de cierto tiempo, elegir la **acción** en  $S$  con **máximo valor estimado**, ejecutarla y volver a empezar desde  $S'$ .

La política para simular trayectorias se llama **política rollout**. Debe ser rápida, para poder hacer muchas simulaciones (ejemplo: aleatoria).

La primera acción  $A$  no se elige según la política rollout. Se simulan trayectorias comenzando en todas las acciones posibles desde  $S$ .

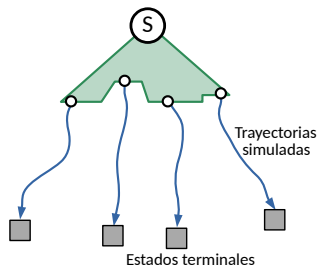
El objetivo es hacer **estimaciones MC de los valores de las acciones desde  $S$** .

(Es como si quisiéramos mejorar la política rollout en  $S$ . Pero esta mejora no se almacena: no son métodos de **aprendizaje** realmente.)



# Decision-time planning: Monte Carlo Tree Search

- Las estimaciones MC de los valores **sí se almacenan** para un **árbol** con raíz en el estado actual  $S$ .
- Ese árbol se va expandiendo incrementalmente.
- Las trayectorias simuladas comienzan en  $S$ , bajan por el árbol y salen por un estado *hoja*.



- Dentro del árbol, seguimos una política que balancea exploración y explotación (p.ej.:  $\epsilon$ -greedy), llamada **política del árbol**.
- Fuera del árbol, completamos las trayectorias siguiendo la **política rollout**.

# Decision-time planning: Monte Carlo Tree Search

1. **Selección:** Siguiendo la **política del árbol** (p.ej.:  $\epsilon$ -greedy), comenzar una trayectoria simulada, desde la raíz hasta una hoja.
2. **Expansión:** En algunas iteraciones, expandir un estado *hoja*, agregando  $1+$  *hijos*.
3. **Simulación:** Completar la trayectoria simulada a partir del estado *hoja* mediante la **política rollout**.
4. **Backup:** Con el retorno generado por la trayectoria simulada, actualizar las estimaciones de los valores de los pares  $(s, a)$  que están dentro del árbol. El resto no se almacenan.
5. **Repetir.**

Luego de cierto tiempo, elegir la **acción** en el estado actual (la *raíz* del árbol) con **máximo valor estimado**. Ejecutarla en el ambiente (real), avanzar en el árbol y volver a empezar. Una parte del árbol puede reusarse, y el resto se descarta.

# Decision-time planning: Monte Carlo Tree Search

**AlphaGo:** MCTS + DNNs (*Rollout policy, Policy network, Value network*)  
entrenadas usando 30M de partidas entre humanos + *Policy gradient*.

2015: 5-0 al campeón europeo Fan Hui.

2016: 4-1 al campeón mundial Lee Sedol.



## ARTICLE

doi:10.1038/nature24270

### Mastering the game of Go without human knowledge

David Silver<sup>1\*</sup>, Julian Schrittwieser<sup>1\*</sup>, Karen Simonyan<sup>1\*</sup>, Ioannis Antonoglou<sup>1</sup>, Aja Huang<sup>2</sup>, Arthur Guez<sup>1</sup>, Thomas Hubert<sup>1</sup>, Lucas Baker<sup>1</sup>, Matthew Lai<sup>1</sup>, Adrian Botros<sup>1</sup>, Yutian Chen<sup>1</sup>, Timothy Lillicrap<sup>1</sup>, Fan Hui<sup>1</sup>, Laurent Sifre<sup>1</sup>, George van den Driessche<sup>1</sup>, Thore Graepel<sup>1</sup> & Demis Hassabis<sup>1</sup>

A long-standing goal of artificial intelligence is an algorithm that learns, *tabula rasa*, superhuman proficiency in challenging domains. Recently, AlphaGo became the first program to defeat a world champion in the game of Go. The tree search in AlphaGo evaluated positions and selected moves using deep neural networks. These neural networks were trained by supervised learning from human expert moves, and by reinforcement learning from self-play. Here we introduce an algorithm based solely on reinforcement learning, without human data, guidance or domain knowledge beyond game rules. AlphaGo becomes its own teacher: a neural network is trained to predict AlphaGo's own move selections and also the winner of AlphaGo's games. This neural network improves the strength of the tree search, resulting in higher quality move selection and stronger self-play in the next iteration. Starting *tabula rasa*, our new program AlphaGo Zero achieved superhuman performance, winning 100-0 against the previously published, champion-defeating AlphaGo.

## ARTICLE

doi:10.1038/nature24861

### Mastering the game of Go with deep neural networks and tree search

David Silver<sup>1\*</sup>, Aja Huang<sup>2\*</sup>, Chris J. Maddison<sup>1</sup>, Arthur Guez<sup>1</sup>, Laurent Sifre<sup>1</sup>, George van den Driessche<sup>1</sup>, Julian Schrittwieser<sup>1</sup>, Ioannis Antonoglou<sup>1</sup>, Veda Fanninchesvaran<sup>1</sup>, Marc Lanctot<sup>1</sup>, Sander Dieleman<sup>1</sup>, Demitris Grewe<sup>1</sup>, John Nham<sup>1</sup>, Nal Kalchbrenner<sup>1</sup>, Ilya Sutskever<sup>1</sup>, Timothy Lillicrap<sup>1</sup>, Madeline Leach<sup>1</sup>, Koray Kavukcuoglu<sup>1</sup>, Thore Graepel<sup>1</sup> & Demis Hassabis<sup>1</sup>

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

### AlphaGo Zero (2017):

MCTS (sin rollouts) + DNN (*Policy+Value network*) entrenada solo mediante *self-play*, sin dataset de partidas entre humanos.

Derrotó 100-0 a AlphaGo.

Aprendió desde *tabula rasa*. (!!)

Descubrió estrategias nuevas, evidenciando sesgos en las teorías humanas del Go.

# Algunas preguntas para las Neurociencias

1. Bandidos con  $k$  brazos: ¿Se sabe cómo eligen los animales entre  $k$  decisiones con recompensas desconocidas? ¿Exploran opciones y luego explotan la mejor?
2. ¿Tiene el cerebro un mecanismo parecido a  $\epsilon$ -greedy para exploración/explotación?
3. ¿Cómo se relaciona la recompensa de RL con la dopamina en el sistema nervioso?
4. ¿Qué fenómenos del comportamiento humano (adicción, toma de riesgo, procrastinación) se podrían (o no) explicar con RL clásico?
5. ¿Es el cerebro un agente que maximiza el retorno, o eso es una simplificación absurda?

# Algunas preguntas para las Neurociencias

6. ¿Qué evidencia hay de que el cerebro representa algo parecido a una tabla  $Q$  o una función de valor?
7. DQN usa *replay de experiencias*. ¿Existe algo análogo en el cerebro? ¿Qué rol podría jugar la consolidación de la memoria durante el sueño?
8. El hipocampo parece fundamental para la planificación y la simulación de futuros. ¿Podría ser una implementación biológica de planificación con *rollouts*, como en Monte Carlo Tree Search?
9. ¿Hay en el cerebro estructuras análogas al *actor* y al *crítico* de los métodos Actor-Critic?
10. ¿Los humanos aprendemos de forma más parecida a AlphaGo (empezando con ciertos datos) o a AlphaGo Zero (*tabula rasa*)?